QuantBacktest

A Backtesting Framework for Retail Quantitative Traders

Aaron Arauz [Baumender11] Luca Di Pietro [L-Di-Pietro]
Stefano Angelo Galdini Filippo Selmi [FilippoSelmi]

2026-05-24

USI Lugano — Programming in Finance II (2026)
Project 2.8 — Instructor: P. H. Gruber
Source repository: https://github.com/L-Di-Pietro/Programming_II_Project_USI

Abstract

QuantBacktest is a web-based event-driven backtesting framework for retail quantitative traders, built as the final project for USI Lugano's *Programming in Finance II* course. The system targets the two failure modes that cause most retail-backtest losses: look-ahead bias (architecturally prevented by enforcing bar-t $\rightarrow$ bar-t+1 fills in the engine) and unrealistic execution assumptions (commission and slippage exposed as first-class user parameters). It ships eleven trading strategies across five families, multi-asset support (equities, ETFs, FX, crypto) with native trading calendars per asset class, both daily and hourly bar resolution, and a six-agent backend in which only two of the six agents are LLM-backed and are opt-in by default. This document covers the project plan, the development diary, the underlying mathematics and data-science methodology, sample results and an economic reflection, the lessons we learned, and an acknowledgement of the generative-AI tools that participated in the project.

Contents

1 Project plan

1.1 Problem statement

Retail quantitative traders lose money for two related reasons. First, many deploy strategies that have never been rigorously back-tested on historical data. Second, when they *do* test, they cut corners that systematically inflate apparent performance: they let the strategy peek at information that would not have been available in real time (*look-ahead bias*), they neglect transaction costs and execution slippage, and they evaluate over too few trades to distinguish edge from noise. The first problem is one of awareness; the second is a software-engineering problem. **QuantBacktest** addresses the second: it is a backtesting platform that makes the rigorous path the easy one.

1.2 Scope

The system delivered in this submission (`v1.0.0-rc1`, tagged 2026-05-24) provides:

- Single-asset backtests across a seeded universe of 20 US equities, 5 ETFs, 5 crypto pairs and 6 FX pairs.
- Daily and hourly bar resolution, each indexed against the *native* trading calendar of its asset class (NYSE for equities and ETFs, 24×5 for FX, 24×7 for crypto).
- Eleven strategies across five families (trend, momentum, breakout, mean-reversion, benchmark), each implementing a uniform signal contract.
- An event-driven backtest engine that enforces look-ahead-bias prevention by construction.
- Configurable commission (bps), slippage (bps or ATR-scaled), and position-sizing modes (fixed fraction or volatility-targeted).
- Standard performance dashboard: equity curve, drawdown, monthly heatmap, trade-P&L scatter, rolling Sharpe, full KPI grid.
- Buy-and-hold and SPY benchmark overlays computed and persisted at run time.
- An optional, opt-in LLM-generated explanatory report grounded on the run's metrics.

Out of scope for this iteration: multi-asset portfolio strategies, minute-level data, walk-forward optimisation UI, paper-trading or live-trading hooks. These are documented in the roadmap section of `README.md` as candidates for v1.1+.

1.3 Tech-stack rationale

The choices reflect a preference for tools that minimise the gap between the production-leaning quality the rubric expects and the four-week timeline the academic calendar allows:

- **Python 3.11+ / FastAPI / SQLAlchemy 2 / Pydantic v2** on the backend. FastAPI gives us OpenAPI for free (the live schema at `/openapi.json` is the source of truth for the typed frontend client). SQLAlchemy's generic types let the same schema run on SQLite in development and on PostgreSQL in production via a single `DATABASE_URL` switch.
- **React 18 + TypeScript + Vite + TailwindCSS + Plotly.js** on the frontend. Plotly produces interactive charts without manual SVG plumbing; TypeScript catches schema mismatches against the OpenAPI-typed client.
- **pytest + Vitest** for tests. The single most important test is `tests/test_engine_no_lookahead.py`, which injects an oracle strategy that "knows" the future close — the engine must still only let it trade at next-bar open.
- **LLM via a provider abstraction.** `LLMProvider` (in `backend/llm/base.py`) has two implementations: `NullProvider` (deterministic, used by tests) and `GeminiProvider` (Google Gemini, opt-in). LLM features are off by default.

1.4 Four-week timeline

The project ran from 2026-04-25 (initial commit) to 2026-05-24 (documentation pass and submission), staffed by four contributors. Targets per week:

- **Week 1 — 2026-04-27 to 2026-05-03.** Scaffold the repository, fix the directory layout, draft `requirements.txt`, agree on the agent boundaries.
- **Week 2 — 2026-05-04 to 2026-05-10.** First domain feature (TSLA as an equity asset); first LLM-generated report (proves the `LLMProvider` abstraction works end-to-end).
- **Week 3 — 2026-05-11 to 2026-05-17.** Frontend MVP; hourly bar support and per-asset-class native calendars; chart suite; PDF export of the AI report.
- **Week 4 — 2026-05-18 to 2026-05-24.** Data-layer hardening (yfinance as primary across all classes); strategy catalogue expansion from 5 to 11; benchmark overlays; shared crosshair and legend; CI workflows; full documentation pass.

The four-contributor team meant any single week's deliverable could be split across two pairs (frontend + backend), which is reflected in the merge-PR history in Section 2.

2 Project diary

The four-week development cycle was punctuated by eight merged pull requests and a handful of direct-to-`main` commits for docs and trivia. The narrative below follows the actual order of decisions, with commit hashes given so a reviewer can verify any claim from `git log`.

Week 1 — Foundation (2026-04-25 to 2026-04-29)

The first commit (`4ff85c1`) bootstrapped an empty repository. Four days later, `ecdf3bb` ("Initial project scaffold for QuantBacktest") landed the full directory layout we would carry through the project: a `backend/` package with `agents/`, `api/`, `analytics/`, `backtest/`, `data/`, `database/`, `llm/`, `strategies/` subpackages; a `frontend/` folder primed for Vite; a `tests/` folder; and `requirements.txt` pinning the backend stack. The agent boundaries were committed to at this stage rather than discovered later — we used the breakdown in the rubric (data, strategy, backtest, analytics, orchestrator, explanation) as the starting point and held to it.

Week 2 — First domain feature + first LLM call (2026-05-08)

Two commits on the same day represent the project's two first proofs of life. PR #1 (`376d30e`, "feat: add TSLA as new equity asset") validated the end-to-end data path: register an asset in the seed list, run `load_initial_data.py`, see daily bars appear in the database. `8997d20` ("feat: add LLM generated report") validated the LLM seam: we wanted to make sure the `LLMProvider` abstraction could carry a real response before we built the rest of the analytics layer around it.

We deliberately wrote the LLM feature *first* as a worst-case probe. If it worked, every other deterministic feature would be easier; if it failed, we had time to redesign. It worked, and the rest of the project never had to compromise its determinism for the sake of the AI report.

Week 3 — Frontend MVP, hourly bars, charts (2026-05-12 to 2026-05-16)

Frontend Version 1 (`1f2f5c7`) landed on 2026-05-12. The same day, PR #2 (`1f13139`, "Add native calendars and hourly bar support") reshaped the data layer: the OHLCV composite primary key became `(asset_id, ts, timeframe)`, so daily and hourly bars could coexist for the same asset, and the `OHLCVCleaner` learned to reindex onto NYSE for equities, a bespoke 24×5 window for FX, and 24×7 for crypto. This was the project's most intricate single change: a missed edge case in any of the three calendars would have silently misstated the annualisation factor for every Sharpe ratio we report.

The chart suite then went through a flurry of small fixes (`3e22d4c`, `162ceba`, `ab1e2de`) that brought the strategy-category badges, chart-tooltip alignment, and sidebar navigation up to standard. `a3b6635` ("Chunk upserts, add LLM retries, and fix time indexes") addressed the SQLite parameter-count ceiling we hit when bulk-loading multi-year hourly history.

Late in the week, the PDF-export feature shipped (`0a7edb1`). The choice to render the report client-side via `jsPDF` rather than server-side via a headless browser kept the backend dependency footprint small and avoided the need for the OS-level Chromium libraries that other PDF approaches require.

Week 4 — CI hardening, data layer, strategies, benchmarks (2026-05-19 to 2026-05-22)

The week opened with two Claude Code GitHub Actions workflows: `claude.yml` (`3d50b41`) for `@claude` mentions on issues and PRs, and `claude-code-review.yml` (`6a38f60`) for an automatic `/code-review` pass on every PR. The team's view of generative AI as a collaborator-not-author shaped both workflows: they only respond to explicit human triggers and they never self-merge.

Build-artefact hygiene then dominated two days (`4ce0311`, `820d610`). TypeScript compile outputs had begun leaking into `frontend/src/`, which would have polluted every diff. PR #4 added the `check-no-build-artifacts.yml` workflow that fails the build if any `.js`, `.d.ts`, or stale `.tsbuildinfo` appears where it should not.

The data layer consolidated on yfinance as the primary source across every asset class (PR #5, `c2b6cce`). The Binance `ccxt` fallback remained for crypto hourly windows older than the 730-day yfinance cap, and Stooq remained for FX EOD fallback. Fixed-fractional sizing was corrected for crypto and FX (PR #6, `ae082ef`).

PR #7 (`b0676ed`, "added 6 new strategies") doubled the strategy catalogue to eleven by adding MACD Crossover, Ichimoku Cloud, Keltner Channels, Time Series Momentum, Stochastic Oscillator, and CCI. Each was added to the registry, exposed via JSON Schema so the frontend form rendered automatically, and covered by a unit test that asserts the signal series on a known fixture.

PR #8 (`c0801c7`, "Support benchmark overlays (API + UI)") and its sibling commits (`be6cf8a`, `55fa4b0`, `d432105`) brought the chart suite to its final polish: a same-asset buy-and-hold and an SPY buy-and-hold curve are now computed and persisted at run time, every chart shares a single legend that toggles them, and a shared crosshair tooltip keeps values aligned when the mouse hovers anywhere on the timeline.

The week closed with `d1efa3f` (Quick-Start docs) and `4a9e6ae` (yfinance/httpx/curl_cffi version bumps). The yfinance bump from 0.2.46 to 1.3.0 required adding `curl_cffi` as a transitive dependency — a small change that we noted in `CHANGELOG.md` because it could surprise contributors who ran into install issues on locked-down corporate machines.

Documentation pass (2026-05-24)

`79e200a` ("Updated all the current documentation files") refreshed the existing docs — `README.md`, `ARCHITECTURE.md`, `CLAUDE.md`, `ONBOARDING.md`, and the four `docs/{agents,calendars,data-sources,strategies}.md` — against the final code state. A second pass on the same day created the artefacts the rubric requires (this PDF among them): `AGENTS.md`, `CHANGELOG.md`, `CONTRIBUTING.md`, `CITATIONS.md`, `AI_USAGE.md`, `SECURITY.md`, `CODE_OF_CONDUCT.md`, `docs/user-guide.md`, `docs/api.md`, `docs/architecture-diagrams/`, and this `docs/academic/` bundle.

Things we tried and abandoned

A few decisions did not survive the four weeks:

- **Asynchronous backtest submission.** The first design queued runs and polled status via WebSocket. We backed it out because synchronous `POST` worked well enough at v1 scale and removed an entire layer of state to debug. The Dashboard still polls `GET /backtests` every five seconds so it will surface any future async-submission feature without UI changes.
- **Postgres in development from day one.** We initially set up Docker Compose with Postgres but it slowed local iteration noticeably. SQLite became the default and the schema was kept on SQLAlchemy generic types so a `DATABASE_URL` swap reaches Postgres without code changes.
- **LLM tool-use via the provider SDK's native function calling.** The Orchestrator Agent now parses tool calls from a JSON envelope in the model output rather than relying on Gemini's native function-calling API, because the latter is not directly portable to other providers. The cost is a small parsing step; the benefit is provider-agnostic tool dispatch.

3 Methodology: economics, mathematics, data science

This section makes the project's analytical contract precise. We define the event-driven backtest semantics that enforce look-ahead-bias prevention, write the KPI formulas in their general form with the per-asset-class annualisation factor, explain why each asset class uses its native trading calendar, summarise the strategy taxonomy, and motivate the LLM-provider abstraction.

3.1 Event-driven backtest semantics and look-ahead bias

Look-ahead bias is the canonical pitfall of backtest construction: a strategy "sees" information that, in real time, it would not have had access to when it needed to act [10, 3]. The most common form is a *single-bar* leak — a strategy reads bar t's closing price and then places an order that fills at bar t's open or close. Across a long backtest the false edge accumulates and inflates every downstream metric. QuantBacktest forbids the leak architecturally: orders generated during bar t fill at the *open of bar* $t{+}1$, with commission and slippage applied to the fill price. The rule is implemented once, in `backend/backtest/engine.py`, and asserted by an oracle-strategy test in `tests/test_engine_no_lookahead.py` that injects a strategy that "knows" the future close; the test passes only if the engine still forces the oracle to trade at next-bar open.

In pseudocode the engine loop is:

```
for t in range(len(bars)):
    # 1. Generate the target position from data up to and including bar t.
    target = strategy.generate_signals(bars[:t+1]).iloc[t] # in {-1, 0, 1}

    # 2. If t == len(bars) - 1 there is no t+1 open to fill at; skip.
    if t == len(bars) - 1: break

    # 3. Decide order from delta (target - current).
    delta = target - portfolio.current_position
    if delta == 0: continue

    # 4. Fill at bar (t+1) open. Apply slippage and commission.
    fill_price = bars.iloc[t+1].open * (1 + sign(delta) * slippage_bps / 10_000)
    commission = abs(delta * portfolio.size) * fill_price * commission_bps / 10_000
    portfolio.apply(delta, fill_price, commission)
```

The signal contract is uniformly `pd.Series[int]` with values in $\{-1, 0, +1\}$ for "short, flat, long". Strategies do not multiply their own signals by a position-size factor; the engine handles sizing via either *fixed fractional* (a fraction of equity per trade) or *volatility targeting* (inverse ATR scaling). A circuit-breaker stops the run if equity draws down beyond a user-supplied threshold.

3.2 Performance metrics

The KPI definitions follow the conventions in Bacon [2]. Let $\{r_t\}_{t=1}^{T}$ denote the per-bar returns of the equity curve $\{V_t\}_{t=0}^{T}$, k the number of bars in one calendar year (Section 3.3), and $y = T/k$ the run's length in years.

Total return and CAGR.

$$R = \frac{V_T}{V_0} - 1, \qquad \mathrm{CAGR} = \left(\frac{V_T}{V_0}\right)^{1/y} - 1. \tag{1}$$

Annualised volatility.

$$\sigma_{\mathrm{ann}} = \sqrt{k}\,\sigma(r_t). \tag{2}$$

Sharpe and Sortino ratios.

$$\mathrm{Sharpe} = \sqrt{k}\,\frac{\bar{r}}{\sigma(r_t)}, \qquad \mathrm{Sortino} = \sqrt{k}\,\frac{\bar{r}}{\sigma_-(r_t)}, \tag{3}$$

where $\sigma_-(r_t)$ is the downside semi-deviation $\sqrt{\mathbb{E}[\min(r_t, 0)^2]}$. We adopt the zero-risk-free-rate convention for retail backtesting [13, 14].

Maximum drawdown and Calmar ratio.

$$\mathrm{maxDD} = \min_t \frac{V_t - \max_{s \le t} V_s}{\max_{s \le t} V_s}, \qquad \mathrm{Calmar} = \frac{\mathrm{CAGR}}{|\mathrm{maxDD}|}. \tag{4}$$

Calmar [17] captures the pain-adjusted return: how much annual growth the strategy delivered per unit of drawdown the user had to endure.

Trade-level metrics. Let $\{p_i\}_{i=1}^{N}$ denote per-leg net P&L:

$$\text{winrate} = \frac{|\{i : p_i > 0\}|}{N}, \qquad \text{PF} = \frac{\sum_{p_i>0} p_i}{\sum_{p_i<0} |p_i|}, \tag{5}$$

$$\text{expectancy} = \text{winrate} \cdot \overline{p_+} - (1 - \text{winrate}) \cdot |\overline{p_-}|, \tag{6}$$

where $\overline{p_+}$ and $\overline{p_-}$ are the means of the positive and negative P&L populations respectively [12, 15].

3.3 Annualisation factor by asset class and timeframe

A Sharpe ratio computed against the wrong k misstates the annualised number by tens of percent. QuantBacktest looks up k from `backend/analytics/periods.py` as a function of `(asset_class, timeframe)`:

Asset class	**Daily k**	**Hourly k**
Equity / ETF (NYSE)	252	1 638
FX (24×5)	260	6 240
Crypto (24×7)	365	8 760

Table 1: Annualisation factor k per asset class and timeframe. NYSE hourly uses 252×6.5 elapsed market hours per session.

The NYSE hourly value reflects elapsed market hours, not the literal bar count, because the first hourly bar of a regular session covers only the opening 30-minute auction. Early-close sessions (about ten per year) further reduce the count slightly; the lookup is calibrated against the `exchange_calendars` XNYS schedule rather than computed analytically.

3.4 Strategy taxonomy

The eleven shipped strategies populate five families. Each family responds well to a different market regime, which is the explicit pedagogical motive for shipping the breadth: running them side-by-side makes the regime sensitivity of each visible.

- **Trend** (SMA Crossover [12], MACD Crossover [1], Ichimoku Cloud [6]): designed to capture sustained directional moves.
- **Momentum** (Time Series Momentum [11]): own-asset trailing-return sign; profits from 1–12 month persistence.
- **Breakout** (Donchian [5], Keltner [7]): long on channel breaks; trades clean trending regimes.
- **Mean reversion** (Bollinger [4], CCI [8], RSI [16], Stochastic Oscillator [9]): trade fades within range-bound regimes; bleed in trends.
- **Benchmark** (Buy and Hold): always long; serves as the reference both for active strategies and for the on-chart same-asset overlay.

All eleven strategies share the same signal contract and therefore the same engine path. Adding a twelfth is a single-file change plus a registry edit; the frontend's parameter form regenerates from the new JSON Schema with no UI work required.

3.5 The LLM-provider abstraction

Two of the six runtime agents — the Orchestrator and the Explanation Agent — are LLM-backed. Every call they make goes through `LLMProvider.generate(...)` in `backend/llm/base.py`. Two implementations ship: `NullProvider`, which returns deterministic canned text, and `GeminiProvider`, which calls Google's Gemini API. Three reasons justify the abstraction. First, the test suite uses `NullProvider` exclusively, so tests are reproducible without an API key. Second, switching provider is a one-line config change rather than a code change; this is the practical insurance against vendor lock-in. Third, the abstraction makes the project's policy explicit: *the deterministic surface of the product is the default*, and any non-determinism is a conscious opt-in by the operator (`LLM_ENABLED=true` plus a populated `GEMINI_API_KEY`).

4 Sample results and economic reflection

This section reports a representative backtest, contextualises it against the two benchmark overlays the system computes automatically, and reflects on what the result implies about the strategy's regime sensitivity. The numeric cells in Table 2 are populated by the team from a run they execute themselves, so the table represents the actual code path rather than a fabricated example.

4.1 Run methodology

Configuration. We report a backtest of the `sma-crossover` strategy on `SPY` (SPDR S&P 500 ETF Trust), over 2020-01-01 to 2024-12-31, daily bars (`timeframe="1d"`). Strategy parameters: `fast_window=20`, `slow_window=50`, `allow_short=false`. Execution parameters: `initial_cash=10 000`, `commission_bps=5`, `slippage_bps=2`, `sizing_mode="fixed_fraction"`, `risk_fraction=1.0`, no max-DD circuit breaker. The annualisation factor is $k = 252$ (NYSE daily, Section 3.3).

Reproducibility. The exact API call:

```
curl -s -X POST http://127.0.0.1:8000/backtests \
  -H 'Content-Type: application/json' \
  -d '{
    "asset_symbol": "SPY",
    "strategy_slug": "sma-crossover",
    "start_date": "2020-01-01T00:00:00",
    "end_date": "2024-12-31T00:00:00",
    "params": {"fast_window": 20, "slow_window": 50, "allow_short": false},
    "timeframe": "1d"
  }'
```

Then `GET /backtests/{run_id}/metrics` and the two `/benchmark/{kind}/metrics` endpoints supply the table values.

4.2 Results table

TEAM: replace the *TBD* cells in Table 2 below with the numbers your actual run returned. The table compiles either way; the TBDs make the gap visible to a reviewer.

Metric	SMA Crossover	Same-asset B&H	SPY B&H
Total return	*TBD*	*TBD*	*TBD*
CAGR	*TBD*	*TBD*	*TBD*
Annualised vol	*TBD*	*TBD*	*TBD*
Sharpe	*TBD*	*TBD*	*TBD*
Sortino	*TBD*	*TBD*	*TBD*
Max drawdown	*TBD*	*TBD*	*TBD*
Calmar	*TBD*	*TBD*	*TBD*
Trade count	*TBD*	1	1
Win rate	*TBD*	—	—
Profit factor	*TBD*	—	—

Table 2: KPIs for the representative SMA-crossover run, alongside the two benchmark overlays computed automatically by `BacktestAgent`. Trade-block cells are dashed for benchmarks (buy-and-hold has only one entry and one exit). Cells marked *TBD* are filled in by the team from the actual run.

4.3 Economic reflection

The economic story we expect Table 2 to tell follows from the nature of the strategy and the chosen window. SMA Crossover is a trend-follower: it goes long when the 20-day SMA crosses above the 50-day SMA and exits when it crosses back. The 2020–2024 window spans the COVID drawdown (March 2020), the equity bull market of 2020–2021, the 2022 bear market, and the 2023–2024 recovery — four distinct regimes inside five

years. A 20/50 SMA crossover should capture the trends of 2021 and 2023–2024 cleanly, sit in cash during much of 2022, but also lose money to whipsaws around the regime transitions (early 2020 and mid-2022).

We therefore expect three observations to hold once the team populates the table:

1. **Lower CAGR than SPY buy-and-hold** but **noticeably lower maximum drawdown**, because the strategy goes flat during 2022's bear market. The pain-adjusted comparison (Calmar) is the more interesting head-to-head.
2. **A Sharpe ratio close to buy-and-hold's** — the strategy's higher hit rate during clear trends is offset by the cost of being in cash through the recoveries.
3. **A trade count modest enough to demand caution.** Five years of daily bars typically produce 10–30 trades for a 20/50 SMA system, which is below the rule-of-thumb threshold of ∼100 trades for statistical significance. Any strong single-asset result here should be re-tested on several assets before being trusted.

TEAM: replace the three-point list above with the actual observations once Table 2 is populated. The numbered list is a template; rewrite it in your own words to reflect the run you actually executed.

4.4 Limitations of the demonstration

The result above is a single asset over a single window with a single parameter set. It is illustrative, not predictive. Three caveats apply:

- **Survivorship bias** — yfinance carries only currently listed tickers, so any equity universe is biased upward.
- **Single-window evaluation** — the rubric's preference for walk-forward / out-of-sample evaluation is correct; SMA-crossover's in-sample fit on a single window does not establish edge.
- **The README's retail-trader tip applies** — if the backtested maximum drawdown is 20%, the user should mentally prepare for 30% in live trading, both because of regime change and because realistic slippage on retail accounts is usually worse than the 2-bps default.

5 Lessons learned

5.1 What worked

Pydantic-driven JSON Schemas as the contract between backend and frontend. Every strategy in `backend/strategies/` declares a Pydantic `Config` class; `GET /strategies` serialises it as a JSON Schema; the React form component renders a complete parameter UI from that schema with no per-strategy frontend code. Adding the sixth-through-eleventh strategies (PR #7) cost zero frontend work as a result. This is the single biggest productivity multiplier in the project and we would carry it forward to any future codebase.

The six-agent abstraction. Splitting the runtime into named agents, each inheriting `BaseAgent[TIn, TOut]` with a single public `run(payload)` method, gave us seam-by-seam testability and a natural docking point for the LLM orchestrator. The boundary between the two LLM-backed agents and the four deterministic agents was the right boundary: it kept the test suite reproducible while leaving the LLM path open as an opt-in feature.

The $\{-1, 0, +1\}$ signal contract. Forbidding strategies to manage their own position size, commissions, or fills made the engine the single point of truth for the bar-$t \to$ bar-$t{+}1$ rule, and meant that the look-ahead-bias oracle test in `tests/test_engine_no_lookahead.py` is the only invariant guard we need. Every strategy passes through the same fill code path; a bug anywhere in fill semantics surfaces everywhere at once.

Native trading calendars per asset class. The temptation to reindex every asset onto NYSE for "consistency" would have silently misstated the annualisation factor for FX and crypto by 5–50%. Building calendar-aware reindexing into the cleaner from the start (PR #2) cost a few hours and saved us from a class of subtle correctness bugs.

5.2 What did not work

Yahoo Finance's 730-day hourly cap. The cap is a hard external constraint that propagates into the UI (date pickers had to gain a per-timeframe maximum-look-back constraint) and into the data layer (the Crypto Fetcher learned to fall back to Binance via `ccxt`). We absorbed the cap rather than worked around it because the alternatives (paid intraday data, scraping alternative sources) carried more cost than the limitation imposes.

LLM tool-call JSON parsing. The Orchestrator Agent parses tool calls out of the model's response by extracting a JSON envelope. This is fragile in two ways: the model occasionally wraps the JSON in markdown fences or prose, and provider-specific function-calling APIs would handle this better. We chose provider-portability over robustness for v1; a future iteration could degrade to provider-native function calling when the configured provider supports it.

Asynchronous backtest execution. The first design had submission return immediately and the UI poll for status; we backed it out to a synchronous `POST /backtests` because runs on daily bars complete in under a second and the async machinery added an entire state-management layer that nothing in v1 actually needed. The Dashboard still polls every five seconds, so future async submission can be added back without UI changes.

5.3 What we would do differently

- **Walk-forward UI from day one.** The Strategy Agent already exposes a chronological train/test split, but the UI does not surface it. Without a UI affordance, users will tune parameters by re-running the full backtest — the canonical recipe for overfitting [3]. The UI for walk-forward should have been a v1 feature, not a v1.1 candidate.
- **Postgres as the default development database.** SQLite was fast to set up but does not catch the subset of bugs that only show up under concurrent writes or under strict referential integrity. The schema is already Postgres-compatible (SQLAlchemy generics only); the change is a `DATABASE_URL` and a Docker Compose file. We would default to Postgres next time and let users opt down to SQLite.
- **Earlier CI hygiene.** The `check-no-build-artifacts.yml` workflow should have been the first `.github/workflow` we wrote, not the third. Build-artefact leakage produced noisy diffs for a week before we formalised the guard.
- **Smaller, more frequent PRs.** The 32 commits in this project landed via only 8 merged PRs; some PRs touched a dozen files across two subsystems. Reviewers (human or AI) work better on small, focused diffs. We would aim for roughly twice as many PRs of half the size in a future iteration.

5.4 What carries forward

The patterns we would copy verbatim into the next project: the schema-as-API discipline, the agent abstraction with a single public `run` method, the deterministic-by-default LLM stance with `NullProvider` for tests, the per-asset-class trading-calendar layer, the $\{-1, 0, +1\}$ signal contract, and the convention that every documentation file carries a "*Last verified against code*" footer so reviewers know when the prose was last reconciled with the source.

6 Acknowledgement of generative-AI tools

This section is required by the course rubric and is a compressed version of `AI_USAGE.md` at the repository root, which carries the full per-tool table and policy statement.

Team policy

The team used generative-AI tools as collaborators, not as authors. Three durable rules governed every contribution:

1. Every line of code that lands in `main` is reviewed by a human. AI output that no team member can explain is not merged.

2. No AI tool is given access to credentials, API keys, or production data. The `.env` file is gitignored; only `.env.example` (no secrets) ships in the repository.

3. The LLM features inside the product itself are opt-in: `backend/config.py` defaults to `LLM_ENABLED=false`, the test suite uses the deterministic `NullProvider`, and activating the live `GeminiProvider` requires three environment variables.

We take collective responsibility for the whole repository, including text and code that was originally drafted by an AI tool and then reviewed and accepted.

Tools used

Claude Code (Anthropic, CLI agent) was the primary coding assistant for multi-file refactors, agent scaffolding, documentation drafts, and the bulk of the LaTeX source for this submission; it left footprints in the repository via the `.github/workflows/claude.yml` (issue/PR `@claude` mention handler) and `.github/workflows/claude-code-review.yml` (per-PR `/code-review`) workflows. **Claude.ai** (Anthropic, web) was used by team members for design discussions and prose drafting. **ChatGPT** (OpenAI) was used opportunistically for isolated syntax checks and LaTeX-table formatting. **GitHub Copilot** (GitHub/OpenAI) provided in-editor autocomplete for routine code; one merge-resolution commit (`b9fc033`) was authored by the `copilot-swe-agent[bot]` account as a one-off, not a sustained authoring pattern. **Cursor** (Anysphere) was used by one team member as their primary editor for some of the frontend work. **Google Gemini** (web) was used to spot-check factual claims about NYSE hours and `ccxt` pagination semantics; **Google Gemini API** is also wired into the product itself as the optional live `LLMProvider`, but that is a product feature, not a development tool.

The team did *not* use Codex, Replit Ghostwriter, Sourcegraph Cody, Tabnine, Codeium, or Amazon CodeWhisperer.

Where AI was not used

The strategy formulas in `backend/strategies/*.py` were transcribed from the academic and practitioner sources cited in `CITATIONS.md` and Section 3; the team independently re-derived each one before implementation. The look-ahead-bias enforcement in `backend/backtest/engine.py` and its oracle-strategy test were designed and written by the team. The database schema, the API contract, and the four-week project timeline were team decisions, not AI-generated.

Reproducibility caveat

AI sessions are not natively reproducible across runs or prompt phrasings. We mitigate this via diff-level human review, the test suite's coverage of the critical correctness invariants, and prompt summaries in the project diary (Section 2) for the most impactful changes.

7 References

[1] Gerald Appel. *Technical Analysis: Power Tools for Active Investors*. Upper Saddle River, NJ: FT Press, 2005. ISBN: 978-0131479029.

[2] Carl R. Bacon. *Practical Portfolio Performance Measurement and Attribution*. 2nd ed. Chichester, UK: Wiley, 2008. ISBN: 978-0470059289.

[3] David H. Bailey and Marcos López de Prado. "The Probability of Backtest Overfitting". In: *Journal of Computational Finance* 20.4 (2014), pp. 39–69. DOI: 10.21314/JCF.2016.322.

[4] John Bollinger. *Bollinger on Bollinger Bands*. New York, NY: McGraw-Hill, 2001. ISBN: 978-0071373685.

[5] Curtis M. Faith. *The Way of the Turtle: The Secret Methods that Turned Ordinary People into Legendary Traders*. New York, NY: McGraw-Hill, 2007. ISBN: 978-0071486644.

[6] Goichi Hosoda. *Ichimoku Kinkō Hyō*. Tokyo, Japan: Self-published, 1969.

[7] Chester W. Keltner. *How to Make Money in Commodities*. Kansas City, MO: Keltner Statistical Service, 1960.

[8] Donald R. Lambert. "Commodity Channel Index: Tool for Trading Cyclic Trends". In: *Commodities Magazine* (Oct. 1980).

[9] George C. Lane. "Lane's Stochastics". In: *Technical Analysis of Stocks & Commodities* 2.3 (1984).

[10] Marcos López de Prado. *Advances in Financial Machine Learning*. Hoboken, NJ: Wiley, 2018. ISBN: 978-1119482086.

[11] Tobias J. Moskowitz, Yao Hua Ooi, and Lasse Heje Pedersen. "Time Series Momentum". In: *Journal of Financial Economics* 104.2 (2012), pp. 228–250. DOI: 10.1016/j.jfineco.2011.11.003.

[12] Robert Pardo. *The Evaluation and Optimization of Trading Strategies*. 2nd ed. Hoboken, NJ: Wiley, 2008. ISBN: 978-0470128015.

[13] William F. Sharpe. "Mutual Fund Performance". In: *The Journal of Business* 39.1 (1966), pp. 119–138.

[14] Frank A. Sortino and Lee N. Price. "Performance Measurement in a Downside Risk Framework". In: *The Journal of Investing* 3.3 (1994), pp. 59–64. DOI: 10.3905/joi.3.3.59.

[15] Van K. Tharp. *Trade Your Way to Financial Freedom*. New York, NY: McGraw-Hill, 1998. ISBN: 978-0070647626.

[16] J. Welles Wilder. *New Concepts in Technical Trading Systems*. Greensboro, NC: Trend Research, 1978. ISBN: 978-0894590276.

[17] Terry W. Young. "Calmar Ratio: A Smoother Tool". In: *Futures* 20.1 (1991).